

完善應用程式內容

- 在本章中，將完成 `command_runner` 套件的最後完善工作。
- 主要任務為改進 `HelpCommand`，使其提供更詳細的使用資訊，並增加 `onOutput` 參數以實現更靈活的輸出處理。
- `CommandRunner` 套件將最終完成，並可用於更複雜的場景。

事前準備工作

- 將 `dartex5` 專案目錄，複製一份成 `dartex6`，做為開發專案資料夾。

```
cp -r dartex5 dartex6
```

- 複製之前，請注意目前所在的工作目錄。

工作開始

- 主要目標：完善 `command_runner` 軟體套件，使其更加強韌且更易於使用。
- Task1: 改進 `HelpCommand` 的輸出
 - 增強 `HelpCommand` 指令，提供更詳細的使用資訊，包括選項及其說明。
 - 這將令使用者更容易理解如何使用命令列應用程式。
 - 首先，開啟 `command_runner/lib/src/help_command.dart` 檔案內容
 - 在檔案開頭新增 `console.dart` 和 `exceptions.dart` 的導入語法。
 - 目的是需要使用顏色延展並拋出 `ArgumentException` 異常。

```
import 'dart:async';

import 'package:command_runner/command_runner.dart';

import 'console.dart';
import 'exceptions.dart';
```

- 請將現有的 `run` 方法改成以下方法。
 - 新版本使用 `StringBuffer` 來有效率地建立協助字串，並包含處理詳細輸出的邏輯。

(以上省略...)

```
@override
FutureOr<String> run(ArgResults args) async {
  final buffer = StringBuffer();
  buffer.writeln(runner.usage.titleText);

  if (args.flag('verbose')) {

    for (var cmd in runner.commands) {
      buffer.write(_renderCommandVerbose(cmd));
    }
  }
}
```

[Back to top](#)

```

    }

    return buffer.toString();
  }

  if (args.hasOption('command')) {

    var (:option, :input) = args.getOption('command');

    var cmd = runner.commands.firstWhere(
      (command) => command.name == input,
      orElse: () {
        throw ArgumentError(
          'Input ${args.commandArg} is not a known command.',
        );
      },
    );

    return _renderCommandVerbose(cmd);
  }

  // Verbose 為 false，且未傳遞任何參數，因此列印基本用法。

  for (var command in runner.commands) {
    buffer.writeln(command.usage);
  }

  return buffer.toString();
}

```

- StringBuffer 是 Dart 的一個類別，可允許有效率地建立字串。
 - 比使用「+」運算子效能更高，尤其是在循環中執行大量字串連接操作時。
- 在 HelpCommand 類別中新增私有輔助方法 _renderCommandVerbose。
 - 此方法用於格式化單一指令的詳細輸出。

(以上省略...)

```

String _renderCommandVerbose(Command cmd) {
  final indent = ' ' * 10;
  final buffer = StringBuffer();

  buffer.writeln(cmd.usage.instructionText);
  buffer.writeln('$indent ${cmd.help}');

  if (cmd.valueHelp != null) {
    buffer.writeln(
      '$indent [Argument] Required? ${cmd.requiresArgument}, Type: $
    );
  }

  buffer.writeln('$indent Options:');
  for (var option in cmd.options) {

```

```

        buffer.writeln('$indent ${option.usage}');
    }
    return buffer.toString();
}

```

- Task2: 新增 onOutput 回傳

- 接下來的工作，是為 CommandRunner 新增 onOutput 參數，以便靈活處理輸出。
- 開啟 command_runner/lib/src/command_runner_base.dart 檔案。
- 將 onOutput 參數加入 CommandRunner 建構函式中，並將對應的 onOutput 成員加入到類別中。

```

class CommandRunner {
  CommandRunner({this.onOutput, this.onError});

  /// 如果此方法不為 null，則用於處理輸出。
  /// 如果希望在輸出列印到控制台之前執行程式碼，
  /// 或者希望執行除了輸出列印到控制台之外的其他操作，則此方法非常有用。

  /// 如果為 null，onInput 方法將 [print] 輸出。

  FutureOr<void> Function(String)? onOutput;

  FutureOr<void> Function(Object)? onError;

  // 以下省略...
}

```

- 更新 run 方法，使其使用 onOutput 參數。

```

Future<void> run(List<String> input) async {
  try {
    final ArgResults results = parse(input);

    if (results.command != null) {
      Object? output = await results.command!.run(results);

      if (onOutput != null) {
        await onOutput!(output.toString());
      } else {
        print(output.toString());
      }
    }
  } on Exception catch (exception) {

    if (onError != null) {
      onError!(exception);
    } else {
      rethrow;
    }
  }
}

```

```
    }
  }
```

- 這會將 run 方法更新為在提供 onOutput 函數時使用該函數，否則預設會列印到控制台。
- Task3: 使用 onOutput 回傳
 - 最後，更新主要程式，用來使用新的 onOutput 功能。
 - 開啟 cli/bin/cli.dart 檔案
 - 更新 main 函數，將 onOutput 函數傳遞給 CommandRunner。
 - 此外，還需要匯入 console.dart 以啟用 write 函數。

```
import 'package:command_runner/command_runner.dart';

const version = '0.0.1';

void main(List<String> arguments) {
  var commandRunner = CommandRunner(
    onOutput: (String output) async {
      await write(output);
    },
    onError: (Object error) {
      if (error is Error) {
        throw error;
      }

      if (error is Exception) {
        print(error);
      }
    },
  )..addCommand(HelpCommand());

  commandRunner.run(arguments);
}
```

- Task4: 測試更新結果
 - 測試改進後的 HelpCommand 和 onOutput 回傳函數。
 - 開啟終端介面並切換到 cli 目錄。
 - 執行下列指令：

```
dart run bin/cli.dart help --verbose
```

- 此時，應該可以看到使用自訂寫入函數列印的 help 指令的詳細用法資訊。

參考文獻

- [Dart 官方教學文件](#)